ESMT
École Supérieure Multinationale des Télécommunications

Base de données NoSQL

# Rapport Du Projet NoSQL:

## Réalisation d'une API RESTful connectée à MongoDB Atlas

| | |
|---|---|
| Étudiant | BOUGOUM Cheick Abdoul Aziz |
| Filière | INGC2 – InDIA |
| Année | 2025 – 2026 |
| Enseignant | M. PREIRA |

# Table des matières


Introduction ..... 4
I. Choix et description du jeu de données ..... 5
1.1 Dataset sélectionné ..... 5
1.2 Justification du choix ..... 5
1.3 Description des champs ..... 5
1.4 Statistiques générales ..... 6
II. Requêtes MongoDB d'interrogation ..... 7
2.1 Requêtes simples ..... 7
R1 — Compter le nombre total de logements ..... 7
R2 — Logements à moins de 50$ par nuit ..... 7
R3 — Compter les appartements entiers à Manhattan ..... 7
R4 — Top 5 hôtes avec le plus de logements ..... 7
R5 — Logements disponibles toute l'année ..... 7
2.2 Requêtes avancées (pipelines d'agrégation) ..... 7
R6 — Prix moyen, min et max par arrondissement ..... 7
R7 — Répartition des types de chambre en pourcentage ..... 8
R8 — Top 5 quartiers avec le plus d'avis ..... 8
R9 — Prix moyen par arrondissement ET par type de chambre ..... 8
R10 — Hôtes les plus actifs avec leurs statistiques ..... 8
III. Déploiement sur MongoDB Atlas ..... 10
3.1 Présentation de MongoDB Atlas ..... 10
3.2 Procédure de déploiement ..... 10
Étape 1 — Création du compte ..... 10
Étape 2 — Création du cluster ..... 10
Étape 3 — Configuration sécurité ..... 11
Étape 4 — Importation du dataset ..... 12
3.3 Connection string ..... 13
3.4 Confirmation de connexion ..... 13
IV. Présentation et explication des codes source ..... 14
4.1 Architecture du projet ..... 14
4.2 server.js — Point d'entrée ..... 15
4.3 config/database.js — Connexion MongoDB ..... 16
4.4 models/Listing.js — Schéma Mongoose ..... 16
4.5 routes/listings.js — Routes REST ..... 16
4.6 .env ..... 18
4.7 scripts/queries.js ..... 19
V. Documentation de l'API ..... 21
5.1 Informations générales ..... 21


INGC2

5.2 Codes de réponse HTTP ..... 21
5.3 Endpoints détaillés ..... 21
GET /api/listings ..... 21
GET /api/listings/stats ..... 21
GET /api/listings/:id ..... 21
POST /api/listings ..... 22
PUT /api/listings/:id ..... 22
DELETE /api/listings/:id ..... 22
VI. Captures d'écran des tests ..... 23
Test 1 — GET /api/listings ..... 23
Test 2 — GET avec filtres ..... 24
Test 3 — GET /stats (agrégation) ..... 24
Test 4 — POST (création) ..... 25
Test 5 — GET par ID ..... 25
Test 6 — PUT (modification) ..... 26
Test 7 — DELETE (suppression) ..... 26
Conclusion ..... 27

INGC2

# Introduction

Dans le cadre du module NoSQL de la formation INGC2 InDIA 2026 à l'ESMT, ce projet a pour objectif la conception et le développement d'une **API RESTful** connectée à une base de données **MongoDB Atlas** hébergée sur le cloud.

Les bases de données NoSQL (Not Only SQL) représentent une alternative aux systèmes relationnels traditionnels. Contrairement aux bases SQL qui organisent les données en tables rigides, MongoDB stocke les informations sous forme de documents JSON flexibles — particulièrement adapté aux données hétérogènes et volumineuses comme celles d'Airbnb.

Le dataset retenu est le **New York City Airbnb Open Data** (Kaggle), recensant 48 895 annonces de logements à New York City pour 2019.

Le projet utilise **Node.js + Express.js** pour le serveur, **Mongoose** comme ODM et **MongoDB Atlas** comme base de données cloud. L'API expose des opérations CRUD complètes ainsi que des requêtes d'agrégation avancées.

Ce document est structuré en six sections : choix du dataset, requêtes MongoDB, déploiement Atlas, architecture du code, documentation API et captures d'écran des tests.

INGC2

# I. Choix et description du jeu de données



*Figure 1: New York City*

## 1.1 Dataset sélectionné

Le dataset retenu est le New York City Airbnb Open Data, disponible sur Kaggle (*https://www.kaggle.com/datasets/dgomonov/new-york-city-airbnb-open-data*). Il recense les annonces de logements Airbnb à New York City pour l'année 2019.

## 1.2 Justification du choix

Ce dataset est particulièrement adapté à MongoDB pour les raisons suivantes :

- Structure documentaire naturelle : chaque logement constitue un document autonome sans jointures complexes.
- Volume significatif : 48 895 documents, idéal pour démontrer les performances des index MongoDB.
- Diversité des types : String, Number, Date — valide le schéma Mongoose.
- Données géographiques : latitude et longitude permettent des analyses spatiales.
- Dataset propre et bien structuré : importable directement via MongoDB Compass.

## 1.3 Description des champs

| Champ | Type | Description |
|---|---|---|
| id | String | Identifiant unique Airbnb du logement |
| name | String | Nom/titre de l'annonce |
| host_id | String | Identifiant unique de l'hôte |
| host_name | String | Prénom de l'hôte |
| neighbourhood_group | String | Arrondissement (Manhattan, Brooklyn, Queens, Bronx, Staten Island) |
| neighbourhood | String | Quartier précis du logement |
| latitude | Number | Coordonnée GPS latitude (décimal) |
| longitude | Number | Coordonnée GPS longitude (décimal) |
| room_type | String | Type de chambre (Entire home/apt, Private room, Shared room) |
| price | Number | Prix par nuit en dollars USD |
| minimum_nights | Number | Nombre minimum de nuits requis |
| number_of_reviews | Number | Nombre total d'avis clients |
| last_review | Date | Date du dernier avis reçu |
| reviews_per_month | Number | Moyenne d'avis reçus par mois |
| calculated_host_listings_count | Number | Nombre total d'annonces publiées par l'hôte sur la plateforme |
| availability_365 | Number | Jours de disponibilité sur 365 jours |

## 1.4 Statistiques générales

- Nombre total de documents : 48 895 logements
- Arrondissements couverts : Manhattan, Brooklyn, Queens, Bronx, Staten Island
- Fourchette de prix : de 0$ à plusieurs milliers de dollars par nuit
- Types de logement : Entire home/apt (52%), Private room (45%), Shared room (3%)

# II. Requêtes MongoDB d'interrogation

## 2.1 Requêtes simples

### R1 — Compter le nombre total de logements

```
Listing.countDocuments();
```

Retourne le nombre total de documents dans la collection. Résultat : 48 895.

### R2 — Logements à moins de 50$ par nuit

```
Listing.find({ price: { $lte: 50 } })
  .select('name price neighbourhood room_type')
  .sort({ price: 1 })
  .limit(5);
```

Filtre avec l'opérateur $lte (less than or equal), tri ascendant, limitation à 5 résultats.

### R3 — Compter les appartements entiers à Manhattan

```
Listing.find({
  neighbourhood_group: 'Manhattan',
  room_type: 'Entire home/apt'
}).countDocuments();
```

Filtre multi-conditions avec ET logique implicite.

### R4 — Top 5 hôtes avec le plus de logements

```
Listing.find()
  .sort({ calculated_host_listings_count: -1 })
  .select('host_name calculated_host_listings_count neighbourhood_group')
  .limit(5);
```

Tri descendant (-1) sur calculated_host_listings_count pour identifier les hôtes professionnels.

### R5 — Logements disponibles toute l'année

```
Listing.countDocuments({ availability_365: 365 });
```

Comptage des logements disponibles les 365 jours de l'année.

## 2.2 Requêtes avancées (pipelines d'agrégation)

### R6 — Prix moyen, min et max par arrondissement

```
Listing.aggregate([
  { $group: {
    _id: '$neighbourhood_group',
    prix_moyen: { $avg: '$price' },
    prix_min:   { $min: '$price' },
    prix_max:   { $max: '$price' },
    total:      { $sum: 1 }
  }},
  { $sort: { prix_moyen: -1 }},
  { $project: {
    quartier: '$_id',
```

```
    prix_moyen: { $round: ['$prix_moyen', 2] },
    prix_min: 1, prix_max: 1, total: 1, _id: 0
  }}
]);
```

Pipeline avec $group, $sort et $project. Utilise $round pour arrondir les moyennes.

### R7 — Répartition des types de chambre en pourcentage

```
Listing.aggregate([
  { $group: { _id: '$room_type', count: { $sum: 1 } }},
  { $project: {
    type: '$_id', count: 1,
    pourcentage: { $round: [{ $multiply: [
      { $divide: ['$count', totalDocs] }, 100] }, 1] },
    _id: 0
  }},
  { $sort: { count: -1 }}
]);
```

Calcul de pourcentage avec $divide et $multiply directement dans le pipeline.

### R8 — Top 5 quartiers avec le plus d'avis

```
Listing.aggregate([
  { $match:  { number_of_reviews: { $gt: 0 } }},
  { $group:  { _id: '$neighbourhood',
    total_avis:   { $sum: '$number_of_reviews' },
    nb_logements: { $sum: 1 } }},
  { $sort:   { total_avis: -1 }},
  { $limit:  5 },
  { $project: { quartier: '$_id', total_avis: 1, nb_logements: 1, _id: 0 }}
]);
```

Pipeline complet : $match → $group → $sort → $limit → $project.

### R9 — Prix moyen par arrondissement ET par type de chambre

```
Listing.aggregate([
  { $match: { price: { $gt: 0, $lt: 500 } }},
  { $group: {
    _id: { quartier: '$neighbourhood_group', type: '$room_type' },
    prix_moyen: { $avg: '$price' },
    count: { $sum: 1 }
  }},
  { $sort: { prix_moyen: -1 }},
  { $limit: 8 },
  { $project: {
    quartier: '$_id.quartier', type: '$_id.type',
    prix_moyen: { $round: ['$prix_moyen', 2] }, _id: 0
  }}
]);
```

Regroupement sur deux champs simultanés avec _id composé.

### R10 — Hôtes les plus actifs avec leurs statistiques

```
Listing.aggregate([
  { $group: {
    _id: '$host_name',
    nb_annonces:  { $sum: 1 },
    prix_moyen:   { $avg: '$price' },
    total_avis:   { $sum: '$number_of_reviews' }
  }},
  { $match:   { nb_annonces: { $gte: 5 } }},
  { $sort:    { nb_annonces: -1 }},
  { $limit:   5 },
  { $project: {
```

```
    hote: '$_id', nb_annonces: 1,
    prix_moyen: { $round: ['$prix_moyen', 2] },
    total_avis: 1, _id: 0
  }}
]);
```

$match après $group — impossible avec find(). Filtre sur le résultat de l'agrégation.

INGC2

# III. Déploiement sur MongoDB Atlas

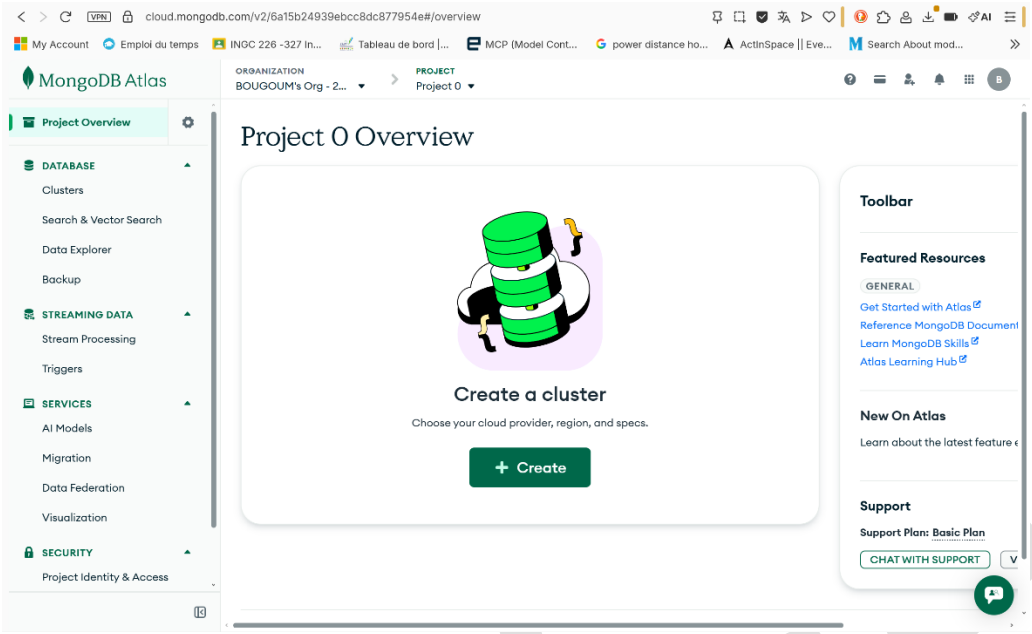



*Figure 2 : MongoDB Atlas*

## 3.1 Présentation de MongoDB Atlas

MongoDB Atlas est la plateforme cloud managée de MongoDB Inc. Le cluster gratuit M0 (512 MB) a été utilisé, suffisant pour héberger les 48 895 documents du dataset NYC Airbnb.

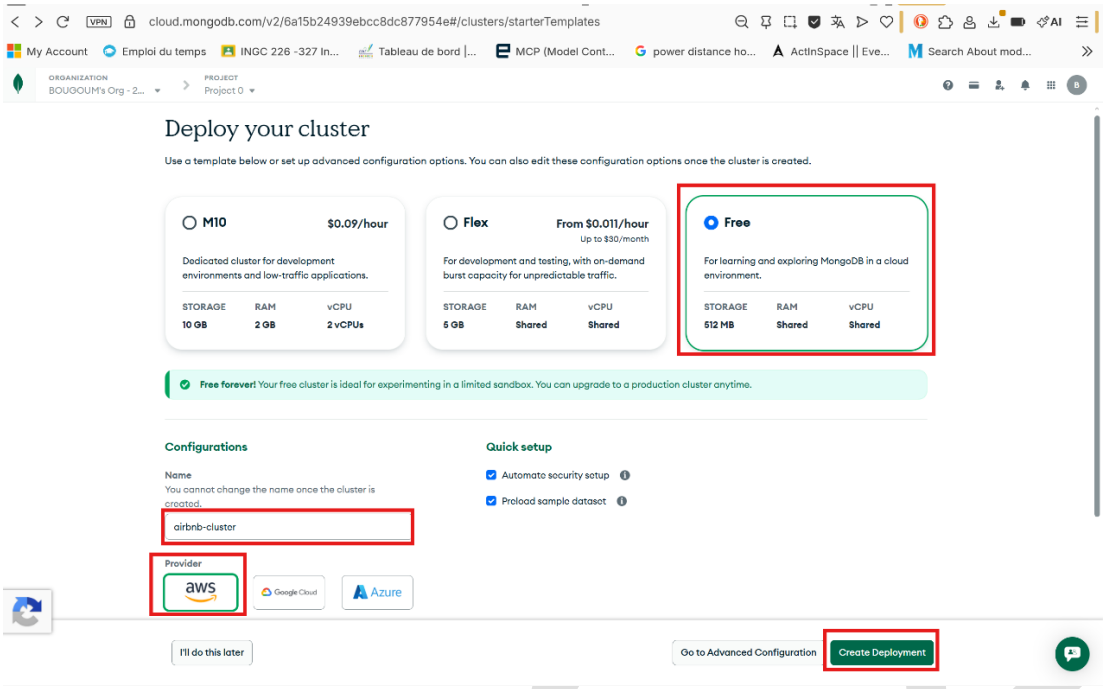
## 3.2 Procédure de déploiement

### Étape 1 — Création du compte

- Inscription sur https://www.mongodb.com/cloud/atlas/register
- Confirmation de l'adresse email

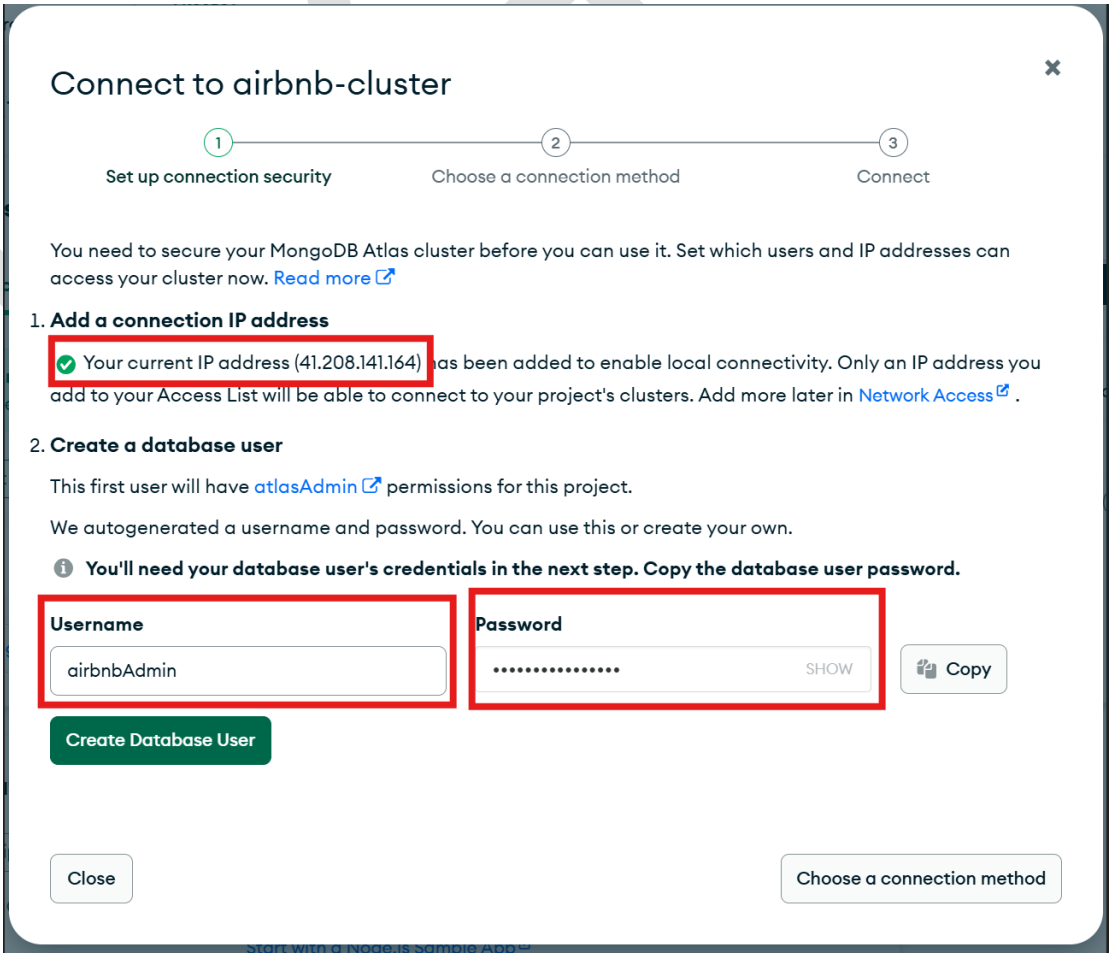
### Étape 2 — Création du cluster

- Plan : Free Tier M0
- Fournisseur : AWS, Région : Europe (Paris)
- Nom : airbnb-cluster — provisionnement en ~2 minutes



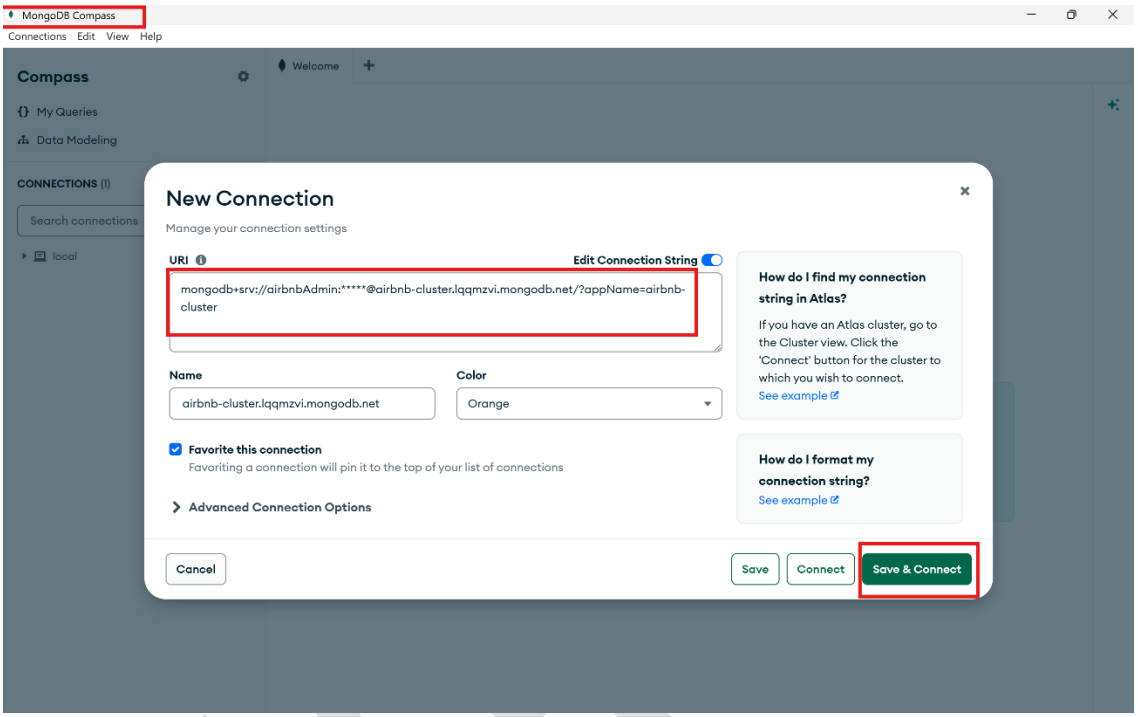

## Étape 3 — Configuration sécurité

- Utilisateur : airbnbAdmin avec mot de passe auto-généré
- Autorisation IP : (adresse IP courrante )

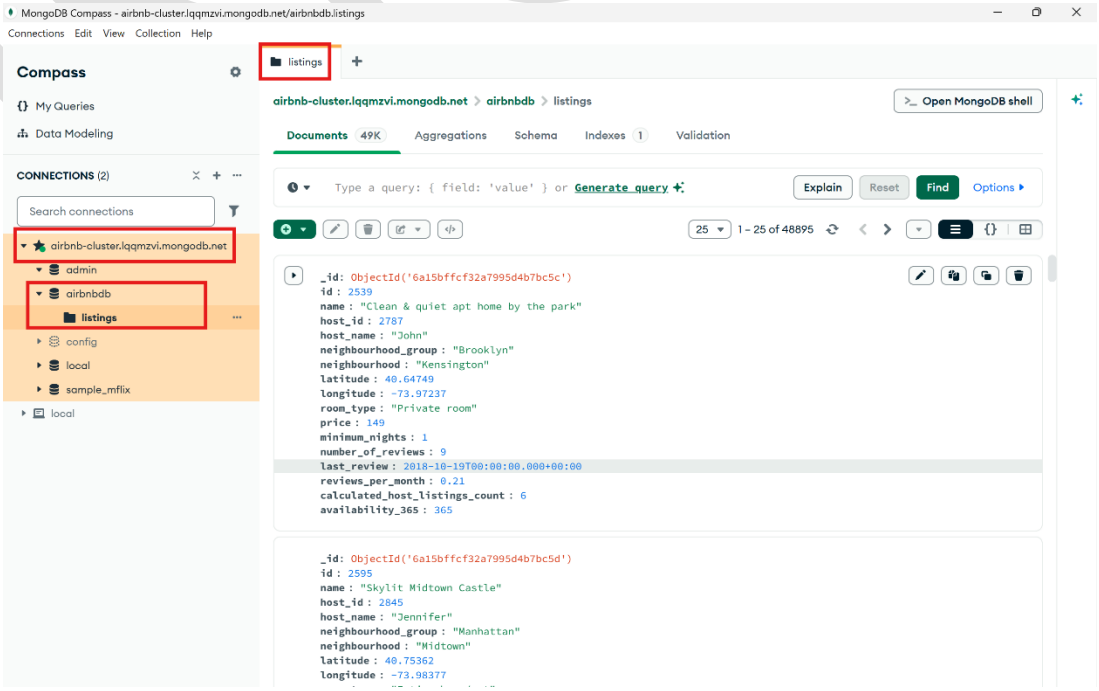


## Étape 4 — Importation du dataset

- Connexion via MongoDB Compass avec la connection string Atlas




- Création de la base airbnbdb, de la collection listings et Import du fichier AB_NYC_2019.csv



## 3.3 Connection string

```
MONGODB_URI=mongodb+srv://airbnbAdmin:<password>@airbnb-cluster
.lqqmzvi.mongodb.net/airbnbdb?retryWrites=true&w=majority
```

Stockée dans le fichier .env (non versionné), chargée par dotenv au démarrage.

## 3.4 Confirmation de connexion

```
☑ Connecté à MongoDB : airbnb-cluster.lqqmzvi.mongodb.net
🚀 Serveur démarré sur http://localhost:3000
```

INGC2

# IV. Présentation et explication des codes source

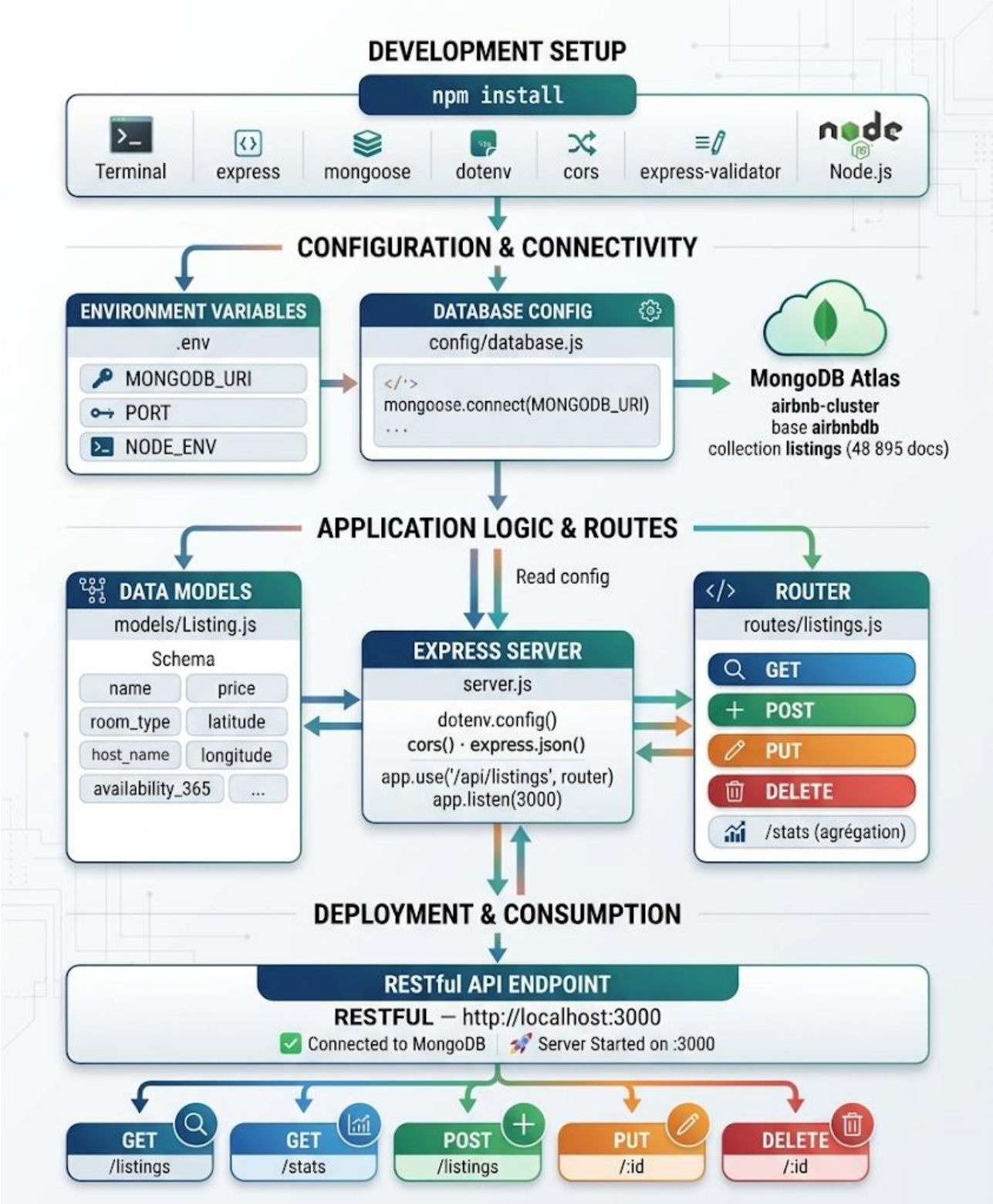



*Figure 3: Architecture de déploiement de l'API RESTful connectée à MongoDB Atlas*

## 4.1 Architecture du projet

| Fichier | Rôle |
|---|---|
| server.js | Point d'entrée : Express, middlewares, routes |
| config/database.js | Connexion MongoDB Atlas via Mongoose |
| models/Listing.js | Schéma Mongoose + index de performance |
| routes/listings.js | 6 routes REST (GET, POST, PUT, DELETE) |
| scripts/queries.js | 10 requêtes MongoDB documentées |
| .env | Variables d'environnement (connection string, port) |

## 4.2 server.js — Point d'entrée

- Chargement dotenv (require('dotenv').config()) — obligatoirement en premier
- Import Express, CORS, connectDB et router
- Appel connectDB() pour établir la connexion Atlas
- Middlewares : cors() et express.json()
- Branchement du router sur /api/listings
- Mise en écoute sur PORT 3000

server.js > ...

```
// server.js
// Point d'entrée — démarre toute l'application

require('dotenv').config();
const express    = require('express');
const cors       = require('cors');
const connectDB  = require('./config/database');
const listings   = require('./routes/listings');

// 1. Connexion à MongoDB Atlas
connectDB();

// 2. Créer l'application Express
const app = express();

// 3. Middlewares
app.use(cors());
app.use(express.json());

// 4. Route de santé — teste que le serveur tourne
app.get('/', (req, res) => {
  res.json({
    message: '🏠 Airbnb API — opérationnelle',
    routes: ['/api/listings', '/api/listings/stats']
  });
});

// 5. Brancher les routes listings
app.use('/api/listings', listings);

// 6. Gérer les routes inexistantes (404)
app.use((req, res) => {
  res.status(404).json({ success: false, message: 'Route introuvable' });
});

// 7. Démarrer le serveur
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`🚀 Serveur démarré sur http://localhost:${PORT}`);
});
```

## 4.3 config/database.js — Connexion MongoDB

- Fonction async connectDB() avec try/catch
- mongoose.connect() avec process.env.MONGODB_URI
- Succès : affichage du host Atlas connecté
- Échec : affichage erreur + process.exit(1)

config > database.js > ...

```
1   // config/database.js
2   // Ce fichier gère la connexion à MongoDB Atlas
3
4   const mongoose = require('mongoose');
5
6   const connectDB = async () => {
7     try {
8       const conn = await mongoose.connect(process.env.MONGODB_URI);
9       console.log(`✅ Connecté à MongoDB : ${conn.connection.host}`);
10    } catch (error) {
11      console.error(`❌ Erreur connexion : ${error.message}`);
12      process.exit(1);
13    }
14  };
15
16  module.exports = connectDB;
```

## 4.4 models/Listing.js — Schéma Mongoose

| Champ | Type | Contraintes |
|---|---|---|
| name | String | required: true, trim: true |
| latitude / longitude | Number | Conversion auto depuis String CSV |
| room_type | String | enum: 4 valeurs, index: true |
| price | Number | min: 0 |
| minimum_nights | Number | min: 1 |
| number_of_reviews | Number | default: 0 |
| calculated_host_listings_count | Number | Nb d'annonces total de l'hôte |
| availability_365 | Number | min: 0, max: 365 |

- Index price (croissant) : accélère les recherches par fourchette de prix
- Index composé neighbourhood_group + price : optimise les filtres par quartier

## 4.5 routes/listings.js — Routes REST

| Méthode | URL | Description |
|---|---|---|
| GET | /api/listings | Liste paginée avec filtres (quartier, type, prix) |
| GET | /api/listings/stats | Statistiques agrégées par arrondissement |
| GET | /api/listings/:id | Récupérer un logement par _id MongoDB |
| POST | /api/listings | Créer un nouveau logement |
| PUT | /api/listings/:id | Modifier partiellement un logement ($set) |
| DELETE | /api/listings/:id | Supprimer définitivement un logement |

routes > listings.js > ...

```
1   // routes/listings.js
2   // Définit toutes les URLs (routes) de l'API
3
4   const express  = require('express');
5   const router   = express.Router();
6   const Listing  = require('../models/Listing');
7
8   // ── ROUTE 1 : Lister tous les logements ──────────────────
9   // GET /api/listings?neighbourhood=Brooklyn&max_price=100
10  router.get('/', async (req, res) => {
11    try {
12      const { neighbourhood, room_type, min_price, max_price, page = 1 } = req.query;
13      const filtre = {};
14      if (neighbourhood) filtre.neighbourhood_group = new RegExp(neighbourhood, 'i');
15      if (room_type)    filtre.room_type = room_type;
16      if (min_price || max_price) {
17        filtre.price = {};
18        if (min_price) filtre.price.$gte = Number(min_price);
19        if (max_price) filtre.price.$lte = Number(max_price);
20      }
21      const limite = 20;
22      const listings = await Listing.find(filtre)
23        .skip((page - 1) * limite)
24        .limit(limite)
25        .sort({ price: 1 });
26      const total = await Listing.countDocuments(filtre);
27      res.json({ success: true, total, page: Number(page), data: listings });
28    } catch (err) {
29      res.status(500).json({ success: false, message: err.message });
30    }
31  });
```

```
// ── ROUTE 2 : Statistiques par quartier ──────────────
// GET /api/listings/stats
router.get('/stats', async (req, res) => {
  try {
    const stats = await Listing.aggregate([
      { $group: {
        _id: '$neighbourhood_group',
        total:     { $sum: 1 },
        prix_moy: { $avg: '$price' },
        prix_min: { $min: '$price' },
        prix_max: { $max: '$price' }
      }},
      { $sort: { prix_moy: -1 }}
    ]);
    res.json({ success: true, data: stats });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// ── ROUTE 3 : Un seul logement par ID ────────────────
// GET /api/listings/:id
router.get('/:id', async (req, res) => {
  try {
    const listing = await Listing.findById(req.params.id);
    if (!listing) return res.status(404).json({ success: false, message: 'Introuvable' });
    res.json({ success: true, data: listing });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

// ── ROUTE 4 : Ajouter un logement ────────────────────
// POST /api/listings
router.post('/', async (req, res) => {
  try {
    const listing = await Listing.create(req.body);
    res.status(201).json({ success: true, data: listing });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
});
```

```
// ── ROUTE 5 : Modifier un logement ───────────────────
// PUT /api/listings/:id
router.put('/:id', async (req, res) => {
  try {
    const listing = await Listing.findByIdAndUpdate(
      req.params.id,
      { $set: req.body },
      { new: true, runValidators: true }
    );
    if (!listing) return res.status(404).json({ success: false, message: 'Introuvable' });
    res.json({ success: true, data: listing });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
});

// ── ROUTE 6 : Supprimer un logement ──────────────────
// DELETE /api/listings/:id
router.delete('/:id', async (req, res) => {
  try {
    const listing = await Listing.findByIdAndDelete(req.params.id);
    if (!listing) return res.status(404).json({ success: false, message: 'Introuvable' });
    res.json({ success: true, message: 'Logement supprimé' });
  } catch (err) {
    res.status(500).json({ success: false, message: err.message });
  }
});

module.exports = router;
```

## 4.6 .env

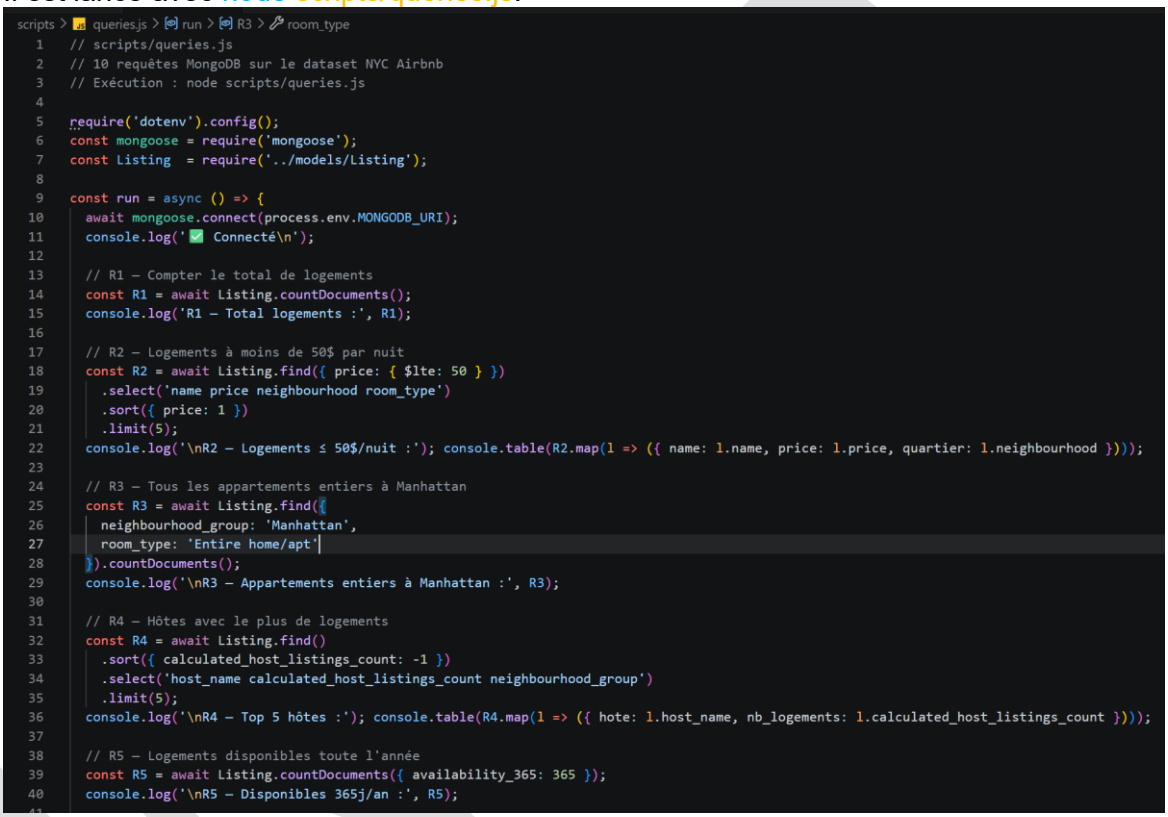
- Stocke les informations sensibles séparément du code source.
- Il n'est jamais partagé ni versionné grâce au .gitignore.

- Chaque variable est chargée au démarrage via require('dotenv').config() dans server.js.
- process.env.MONGODB_URI donne accès à la connection string dans n'importe quel fichier du projet sans jamais exposer le mot de passe dans le code.

```
MONGODB_URI=mongodb+srv://airbnbAdmin:<password>@airbnb-
cluster.lqqmzvi.mongodb.net/airbnbdb?retryWrites=true&w=majority
PORT=3000
NODE_ENV=development
```

## 4.7 scripts/queries.js

- Il se connecte directement à Atlas, exécute les 10 requêtes,
- affiche les résultats dans le terminal puis se déconnecte.
- Il est lancé avec node scripts/queries.js.



```
scripts > queries.js > run > R3 > room_type
// scripts/queries.js
// 10 requêtes MongoDB sur le dataset NYC Airbnb
// Exécution : node scripts/queries.js

require('dotenv').config();
const mongoose = require('mongoose');
const Listing  = require('../models/Listing');

const run = async () => {
  await mongoose.connect(process.env.MONGODB_URI);
  console.log('✅ Connecté\n');

  // R1 — Compter le total de logements
  const R1 = await Listing.countDocuments();
  console.log('R1 — Total logements :', R1);

  // R2 — Logements à moins de 50$ par nuit
  const R2 = await Listing.find({ price: { $lte: 50 } })
    .select('name price neighbourhood room_type')
    .sort({ price: 1 })
    .limit(5);
  console.log('\nR2 — Logements ≤ 50$/nuit :'); console.table(R2.map(l => ({ name: l.name, price: l.price, quartier: l.neighbourhood })));

  // R3 — Tous les appartements entiers à Manhattan
  const R3 = await Listing.find({
    neighbourhood_group: 'Manhattan',
    room_type: 'Entire home/apt'
  }).countDocuments();
  console.log('\nR3 — Appartements entiers à Manhattan :', R3);

  // R4 — Hôtes avec le plus de logements
  const R4 = await Listing.find()
    .sort({ calculated_host_listings_count: -1 })
    .select('host_name calculated_host_listings_count neighbourhood_group')
    .limit(5);
  console.log('\nR4 — Top 5 hôtes :'); console.table(R4.map(l => ({ hote: l.host_name, nb_logements: l.calculated_host_listings_count })));

  // R5 — Logements disponibles toute l'année
  const R5 = await Listing.countDocuments({ availability_365: 365 });
  console.log('\nR5 — Disponibles 365j/an :', R5);
```

scripts > queries.js > run > R3 > room_type

```
9   const run = async () => {
41
42    // R6 — Prix moyen, min, max par quartier
43    const R6 = await Listing.aggregate([
44      { $group: {
45        _id: '$neighbourhood_group',
46        prix_moyen: { $avg: '$price' },
47        prix_min:   { $min: '$price' },
48        prix_max:   { $max: '$price' },
49        total:      { $sum: 1 }
50      }},
51      { $sort: { prix_moyen: -1 }},
52      { $project: {
53        quartier:   '$_id',
54        prix_moyen: { $round: ['$prix_moyen', 2] },
55        prix_min: 1, prix_max: 1, total: 1, _id: 0
56      }}
57    ]);
58    console.log('\nR6 — Prix par quartier :'); console.table(R6);
59
60    // R7 — Répartition des types de chambre en %
61    const totalDocs = await Listing.countDocuments();
62    const R7 = await Listing.aggregate([
63      { $group: { _id: '$room_type', count: { $sum: 1 } }},
64      { $project: {
65        type: '$_id',
66        count: 1,
67        pourcentage: { $round: [{ $multiply: [{ $divide: ['$count', totalDocs] }, 100] }, 1] },
68        _id: 0
69      }},
70      { $sort: { count: -1 }}
71    ]);
72    console.log('\nR7 — Répartition types de chambre :'); console.table(R7);
73
74    // R8 — Top 5 quartiers avec le plus d'avis
75    const R8 = await Listing.aggregate([
76      { $match:  { number_of_reviews: { $gt: 0 } }},
77      { $group:  { _id: '$neighbourhood', total_avis: { $sum: '$number_of_reviews' }, nb_logements: { $sum: 1 } }},
78      { $sort:   { total_avis: -1 }},
79      { $limit:  5 },
80      { $project: { quartier: '$_id', total_avis: 1, nb_logements: 1, _id: 0 }}
81    ]);
82    console.log('\nR8 — Top 5 quartiers par avis :'); console.table(R8);
83
```

scripts > queries.js > run > R3 > room_type

```
9   const run = async () => {
84    // R9 — Prix moyen par type ET par quartier
85    const R9 = await Listing.aggregate([
86      { $match: { price: { $gt: 0, $lt: 500 } }},
87      { $group: {
88        _id: { quartier: '$neighbourhood_group', type: '$room_type' },
89        prix_moyen: { $avg: '$price' },
90        count: { $sum: 1 }
91      }},
92      { $sort: { prix_moyen: -1 }},
93      { $limit: 8 },
94      { $project: {
95        quartier: '$_id.quartier', type: '$_id.type',
96        prix_moyen: { $round: ['$prix_moyen', 2] }, count: 1, _id: 0
97      }}
98    ]);
99    console.log('\nR9 — Prix moyen par quartier ET type :'); console.table(R9);
100
101   // R10 — Hôtes les plus actifs avec leurs stats
102   const R10 = await Listing.aggregate([
103     { $group: {
104       _id: '$host_name',
105       nb_annonces:  { $sum: 1 },
106       prix_moyen:   { $avg: '$price' },
107       total_avis:   { $sum: '$number_of_reviews' }
108     }},
109     { $match:   { nb_annonces: { $gte: 5 } }},
110     { $sort:    { nb_annonces: -1 }},
111     { $limit:   5 },
112     { $project: {
113       hote: '$_id', nb_annonces: 1,
114       prix_moyen: { $round: ['$prix_moyen', 2] },
115       total_avis: 1, _id: 0
116     }}
117   ]);
118   console.log('\nR10 — Top hôtes actifs :'); console.table(R10);
119
120   await mongoose.disconnect();
121   console.log('\n✅ Toutes les requêtes exécutées.');
122 };
123
124 run().catch(err => { console.error(err); process.exit(1); });
```

# V. Documentation de l'API

## 5.1 Informations générales

| | |
|---|---|
| URL de base | http://localhost:3000 |
| Format | JSON (application/json) |
| Authentification | Aucune (projet académique) |
| Version | 1.0.0 |

## 5.2 Codes de réponse HTTP

| Code | Signification |
|---|---|
| 200 OK | Requête réussie — données retournées |
| 201 Created | Ressource créée avec succès (POST) |
| 400 Bad Request | Données invalides — erreur de validation Mongoose |
| 404 Not Found | Logement introuvable avec l'ID fourni |
| 500 Server Error | Erreur serveur interne |

## 5.3 Endpoints détaillés

### GET /api/listings

Paramètres (query string) :

| Paramètre | Type | Description |
|---|---|---|
| neighbourhood | String | Filtre arrondissement (insensible casse) |
| room_type | String | Type de chambre |
| min_price | Number | Prix minimum par nuit |
| max_price | Number | Prix maximum par nuit |
| page | Number | Numéro de page (défaut: 1, 20 résultats/page) |

```
GET http://localhost:3000/api/listings?neighbourhood=Brooklyn&max_price=80
```

### GET /api/listings/stats

```
GET http://localhost:3000/api/listings/stats
```

### GET /api/listings/:id

```
GET http://localhost:3000/api/listings/6a15c00bf32a7995d4b816d5
```

### POST /api/listings

Corps JSON requis :

```
{ "name": "Studio moderne", "host_name": "Jean",
  "neighbourhood_group": "Brooklyn", "room_type": "Private room",
  "latitude": 40.7128, "longitude": -74.0060, "price": 75 }
```

### PUT /api/listings/:id

```
{ "price": 90, "minimum_nights": 3 }
```

Seuls les champs envoyés sont modifiés grâce à l'opérateur $set.

### DELETE /api/listings/:id

```
DELETE http://localhost:3000/api/listings/:id
```

INGC2

N

# VI. Captures d'écran des tests

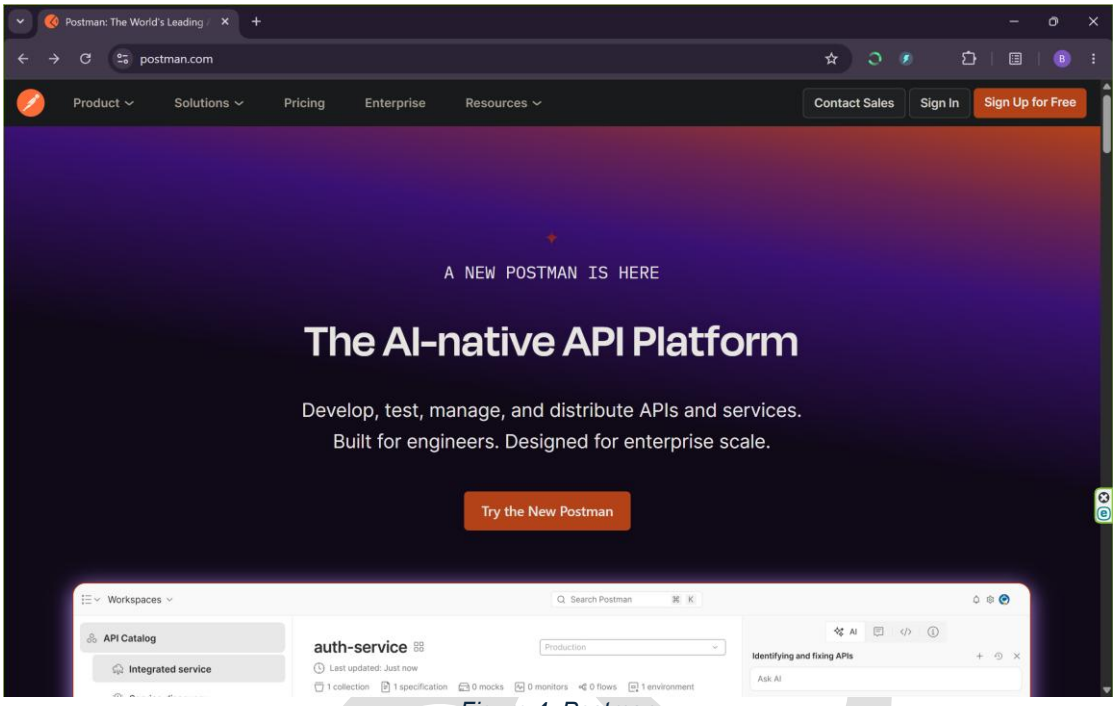
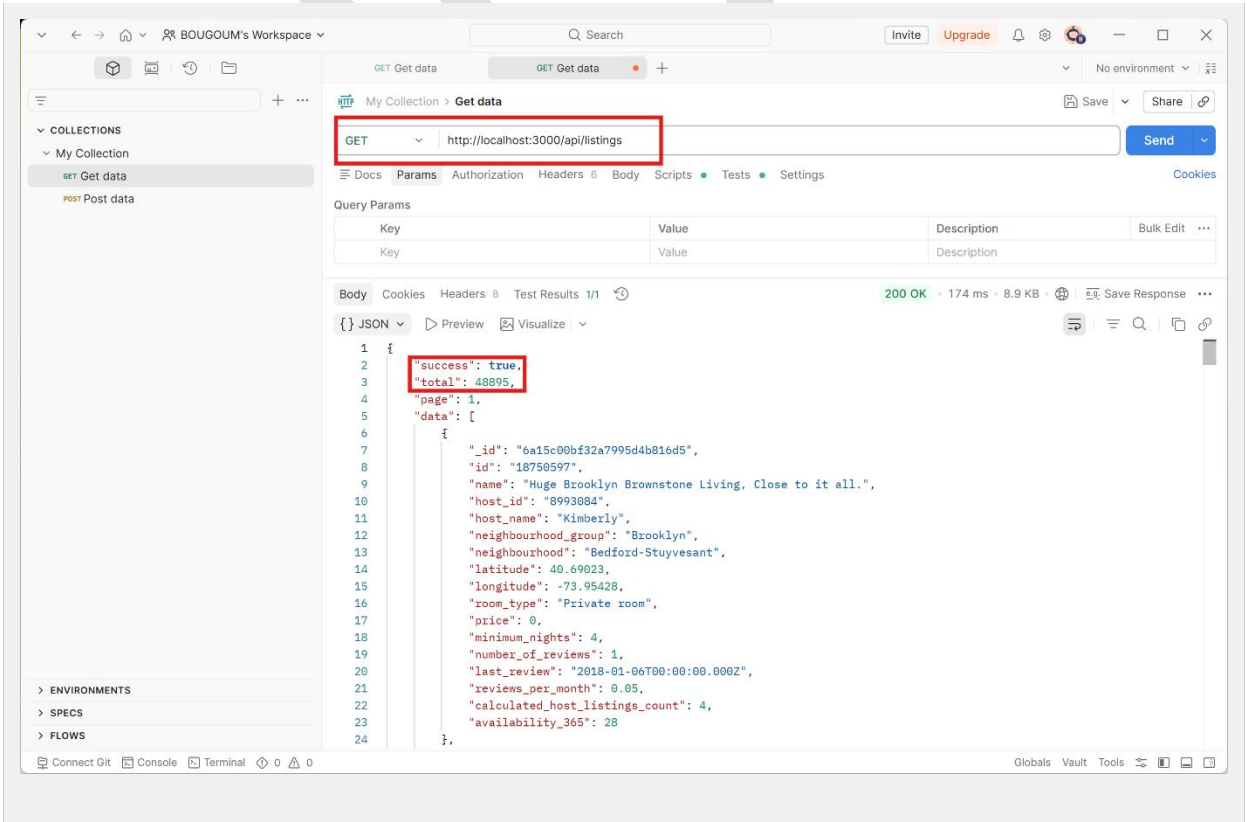



*Figure 4: Postman*

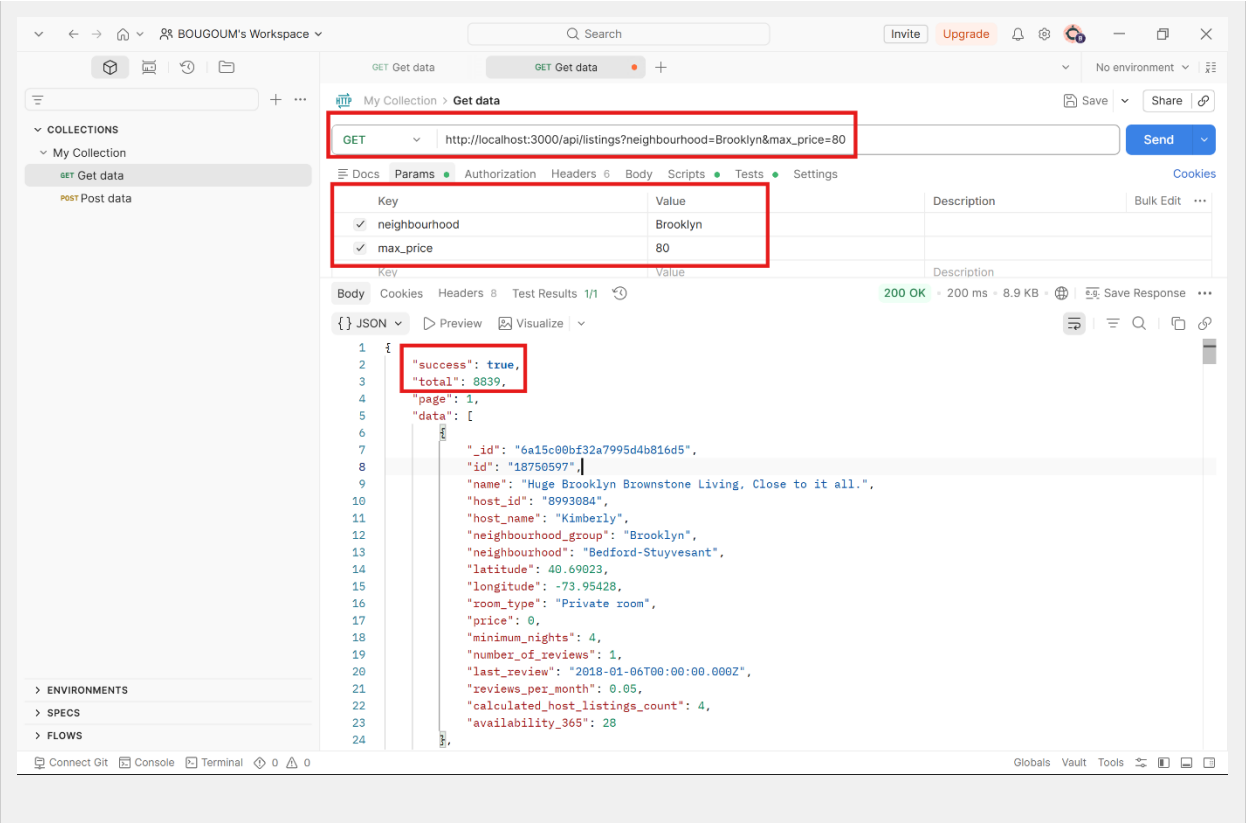
## Test 1 — GET /api/listings

200 OK — Retourne 48 895 logements paginés (20/page)



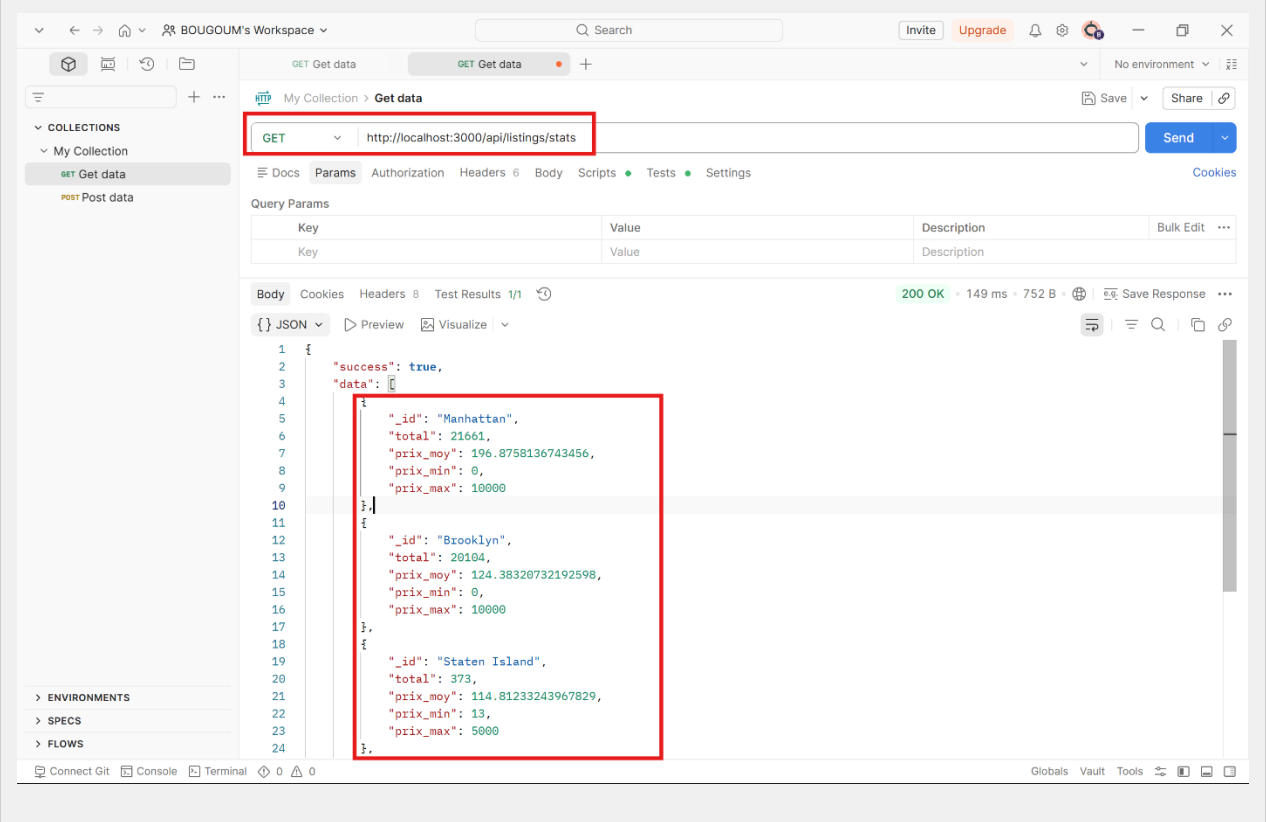
## Test 2 — GET avec filtres

200 OK — Logements Brooklyn ≤ 80$/nuit, triés par prix



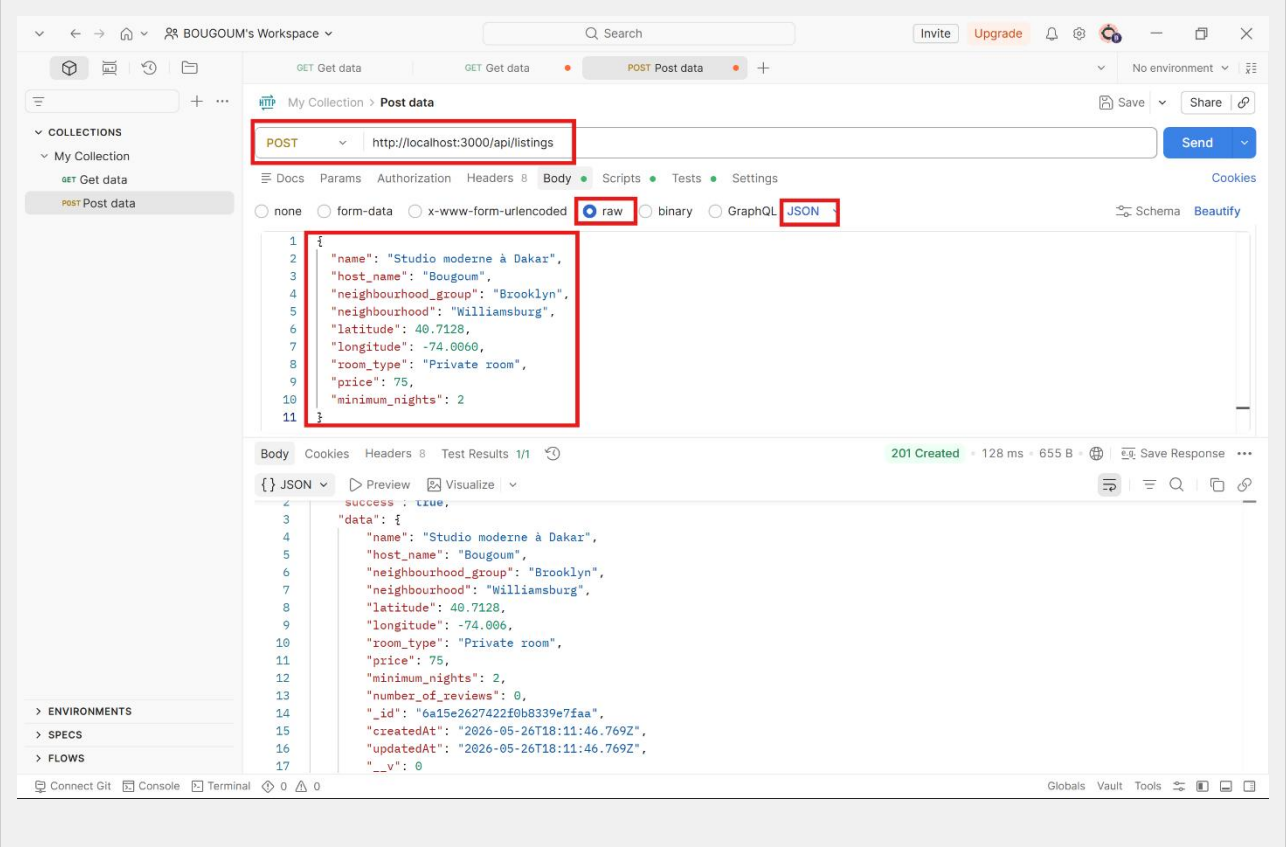

## Test 3 — GET /stats (agrégation)

200 OK — Prix moyen/min/max par arrondissement via pipeline $group



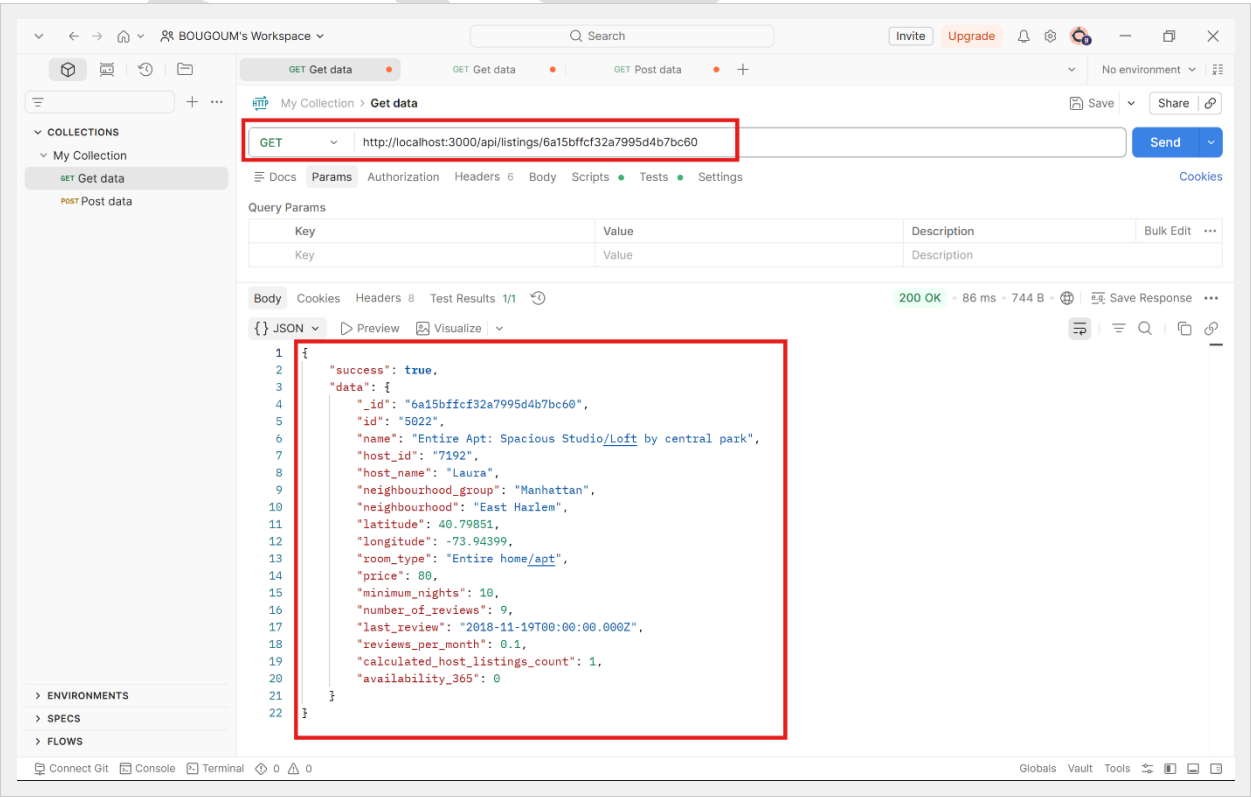
## Test 4 — POST (création)

201 Created — Nouveau document créé avec _id MongoDB auto-généré



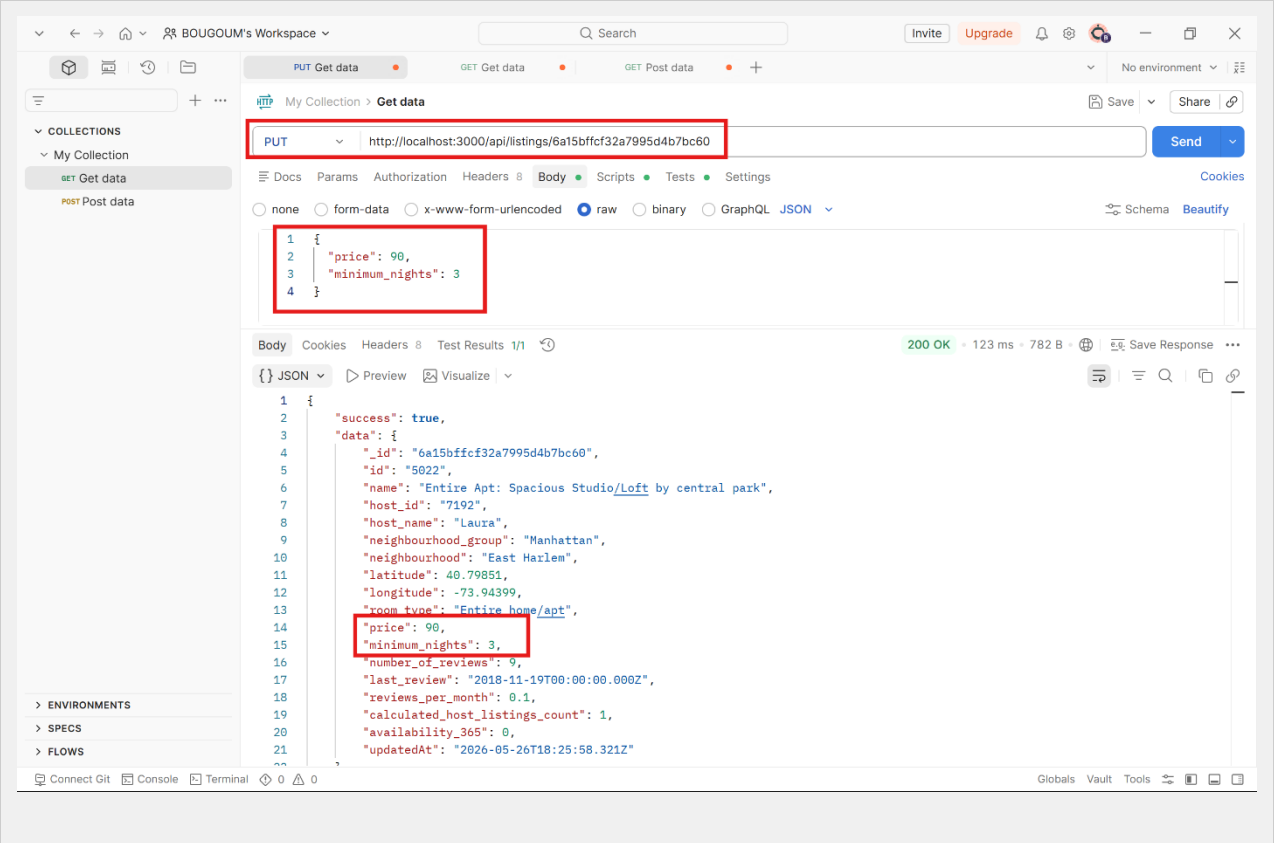

## Test 5 — GET par ID

200 OK — Document complet retourné par _id



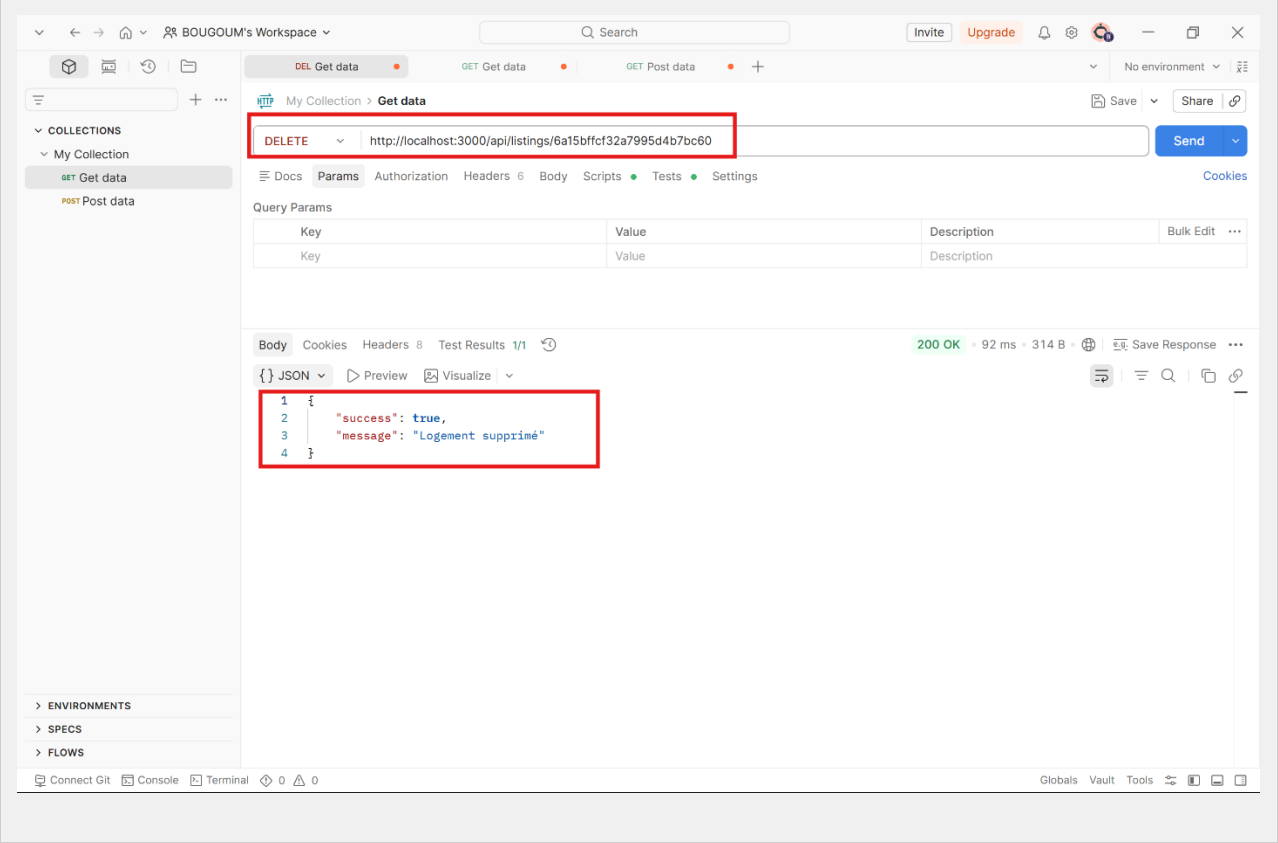
## Test 6 — PUT (modification)

200 OK — price et minimum_nights modifiés, autres champs intacts




## Test 7 — DELETE (suppression)

200 OK — Document supprimé définitivement de Atlas




# Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux des bases de données NoSQL au travers d'une réalisation concrète et fonctionnelle. L'implémentation d'une API RESTful complète, connectée à MongoDB Atlas, a couvert l'ensemble du cycle de développement : de la conception du schéma jusqu'aux tests avec Postman.

Les principaux apprentissages de ce projet sont :

- La flexibilité du modèle document de MongoDB, qui s'adapte naturellement aux données hétérogènes comme celles d'Airbnb, sans les contraintes des schémas relationnels rigides.
- La puissance des pipelines d'agrégation ($group, $match, $sort, $project) pour réaliser des analyses statistiques directement en base de données.
- L'importance des index MongoDB pour optimiser les performances sur un volume de 48 895 documents.
- La simplicité de déploiement de MongoDB Atlas qui abstrait la gestion de l'infrastructure cloud.
- L'architecture REST avec Node.js/Express et Mongoose, offrant une séparation claire entre les couches de l'application.

En perspective, ce projet pourrait être enrichi par l'ajout d'une authentification JWT pour sécuriser les endpoints, par des requêtes géospatiales exploitant les coordonnées GPS du dataset, ou par le déploiement de l'API sur une plateforme cloud publique comme Render ou Heroku.

Ce projet démontre ainsi comment les bases de données NoSQL et les technologies JavaScript modernes peuvent être combinées pour construire des APIs performantes, scalables et bien documentées.